MOSIP Decode 2026 -- Problem Statement 1

Automated Conformance Testing for Inji Certify and Inji Verify

Project Handbook

Shardul Chogale (conformance engine and evidence)
Mukta Varak (platform and delivery)

Written: 30 September 2026; updated 2 October 2026
Final submission deadline: **2 November 2026** (extended from 26 October)

Part 1 Mentor session brief -- Weekly Connect, 7 October
Part 2 Start to end: the clean guide to running the project
Part 3 Record of every step taken so far
Part 4 Planner, to 2 November -- final week re-planned 26 October

This is the document to open first. It is kept current: when a procedure changes, Part 2 changes with it; when a day's work is done, Part 3 gains an entry and Part 4 gets its boxes ticked. The detailed evidence behind every claim lives in docs/findings/findings.tex.

Contents

Part 1 Mentor session brief . . . 3
1.1 Where we are, in one paragraph . . . 3
1.2 Three things to show if asked . . . 3
1.3 Questions, ranked by how much the answer changes the build . . . 3
1.4 What we want to leave with . . . 5
1.5 Notes during the session . . . 5
Part 2 Start to end: running the project from a clean machine . . . 6
2.1 What runs, and why each piece is there . . . 6
2.2 Step 1 -- Prerequisites . . . 6
2.3 Step 2 -- The working directory . . . 6
2.4 Step 3 -- ngrok, once . . . 7
2.5 Step 4 -- Stack settings . . . 7
2.6 Step 5 -- Point the runner at your machine . . . 7
2.7 Step 6 -- Start the stack . . . 7
2.8 Step 7 -- Check everything is healthy . . . 8
2.9 Step 8 -- Run the unit tests . . . 8
2.10 Step 9 -- Run conformance . . . 8
2.11 Step 10 -- Verify the results . . . 8
2.12 Step 11 -- Record what the run showed . . . 9
2.13 Step 12 -- Commit . . . 9
2.14 Step 13 -- Shut down . . . 9
2.15 Step 14 -- Ending the project . . . 9
2.16 Troubleshooting -- problems that have already cost us time . . . 10
Part 3 Record of every step taken . . . 11
3.1 16-20 September -- registration and pre-kickoff groundwork . . . 11
3.2 23 September -- official kickoff: plan, roles, contract . . . 11
3.3 24 September -- the runner . . . 11
3.4 29 September -- first live runs . . . 11
3.5 30 September -- deadline found, this handbook . . . 12
3.6 2 October -- the kickoff recording, and the 1.0 line . . . 12
3.7 26 October -- deadline extended; the api-testrig class . . . 12
Part 4 Planner, to 2 November . . . 13
4.1 Re-plan, 26 October: the final week . . . 13
4.2 Fixed points . . . 13
4.3 Week 2 remainder -- Wed 30 Sep to Sun 4 Oct: move to the 1.0 line . . . 14
4.4 Remaining effort -- Shardul's part (estimate, 2 October) . . . 14
4.5 Week 3 -- Mon 5 to Sun 11 Oct: the two halves meet . . . 15
4.6 Week 4 -- Mon 12 to Sun 18 Oct: complete the mandatory set . . . 15
4.7 Week 5 -- Mon 19 to Mon 26 Oct: submit . . . 15
4.8 Dependencies between us . . . 16

4.9 Risks, and what we do if they land 16

Part 1 Mentor session brief

Session	MOSIP Decode Weekly Connect
When	Wednesday 7 October 2026, 5:00-6:00 PM IST
Where	https://mosip.zoom.us/j/89100675963
Attending	Shardul (and Mukta, if available)

1.1 Where we are, in one paragraph

We have a working harness that drives the OpenID Foundation conformance suite's REST API against Inji Verify with no human interaction. One command -- `./run-conformance.sh -component verify` -- creates the test plan, starts each module, obtains an authorisation request from Inji Verify's own API, delivers it to the suite, collects the result, and gates it against a recorded baseline. On 29 September its first unattended run reproduced our Day 1 manual run **check for check**: 15 passing checks, 4 failures, 3 warnings, the identical set of seven non-passing checks, and the same point of termination. Versions tested: Inji Verify 0.18.2, conformance suite 5.3.1. Inji Certify is not yet running.

Since then (2 October): we found that upstream has an Inji **1.0 line** (release-1.0.x, published as 1.0.0-alpha.1) -- which is what the problem statement means by "on 1.0". On it, DCQL is implemented and the nonce is random, so our three headline findings do not apply there. We are moving the baseline to 1.0.0-alpha.1; the findings stand for 0.18.2 and are on hold for upstream reporting.

1.2 Three things to show if asked

1. **The automated run matches the manual run exactly.** This is what makes everything else credible: the harness measures what a tester would see by hand.
2. **The nonce experiment.** Two runs differing only in where the nonce comes from. With the nonce the Verify UI's SDK sends, the entropy check warns (71.68 bits against 96 required) and the URL-safety check fails. When verify-service generates the nonce itself, both pass and nothing else changes. The weak nonce is btoa(Date.now()) in the SDK; the service already does it correctly.
3. **The benchmark gate.** Inji Verify fails every module of the OID4VP 1.0 Final plan at DCQL extraction, so the gate fails a build on *change from the recorded baseline*, not on absolute pass rate. The suite's own verdict is never rewritten -- only the TestNG verdict is gated.

1.3 Questions, ranked by how much the answer changes the build

Each question has a default we will follow if there is no answer, so nothing blocks on the session. The top three are the ones worth the time.

#	Question	Why it matters	Our default if unanswered
1	**Q1 -- Which spec version is the benchmark?** Verify 0.18.2 implements Presentation Exchange; OID4VP 1.0 Final requires DCQL.	Decides whether any module can pass, and what the gate measures. Every 1.0 Final module currently stops at DCQL extraction.	1.0 Final. Since the 1.0 line implements DCQL, this is now the natural benchmark rather than an all-fail one; the ID2 plan drops to optional.

#	Question	Why it matters	Our default if unanswered
2	**Confirm the 1.0 target.** We found release-1.0.x (Verify and Certify) and the published 1.0.0-alpha.1; alpha.2 is named in the branch but not yet published. Is 1.0.0-alpha.1 the intended target, or should we build the branch from source? Is the 0.18.x line still maintained?	Decides what our baseline is, and whether the 0.18.2 findings are worth reporting at all.	Target 1.0.0-alpha.1; hold the 0.18.2 findings unless told the line is maintained.
3	**Q7 -- Is there a lighter Inji Certify setup?** docker-compose-injistack needs a PKCS12 keystore from Mimoto onboarding, a public DID endpoint, an external authorisation server and partner API keys.	Decides whether the combined run is achievable locally. Certify's issuer plan is 61 modules against the verifier plan's 12.	Attempt the local stack; if blocked, submit Verify end to end plus a documented Certify path, and say so plainly.
4	**Where should findings go upstream?** GitHub issues on mosip/inji-verify, or another tracker? Would a pull request fixing the SDK nonce be welcome from us?	This is our open-source contribution path. Our strongest finding is a three-line fix.	A GitHub issue on mosip/inji-verify, followed by a pull request from our fork.
5	**Q4 -- Benchmark semantics.** Regression-from-baseline, or an absolute target the build fails until conformance improves?	Changes what "pass" means in CI.	Regression-from-baseline (built). Absolute is one flag: -policy absolute-pass-rate.
6	**Submission criteria.** The kickoff settled the list -- source code, working prototype, slide deck, video demo. Remaining: must the api-testrig classes be merged upstream, or are branches on our forks acceptable?	Scopes the last week.	Deliver the Java classes as fork branches shaped as upstream pull requests.
7	**Q2 -- HAIP.** Verify hard-codes direct_post; HAIP requires direct_post.jwt. Track HAIP as a known-failing suite, or exclude it?	Affects scope, not architecture.	Exclude from the gate; document the gap.
8	**Q3 -- Client identifier schemes.** Verify offers did and pre_registered; the 1.0 Final plan offers redirect_uri, x509_san_dns, x509_hash. No overlap.	Explains one of the failures in every run.	Record as a known gap.

#	Question	Why it matters	Our default if unanswered
9	**Q5 / Q8 -- Configuration questions.** Is the shipped config.json meant to represent a certifiable deployment? Does anything rely on the presentation definition id, which the UI hard-codes?	Decides whether two observations become findings.	Keep both as observations, not findings.
--	**Q6 -- answered by us; confirm only.** Unattended runs work: the harness delivers the request by HTTP GET to the test's exposed authorization_endpoint, which is what the suite's paste box does internally.	Confirms the approach is acceptable.	Proceed.

1.4 What we want to leave with

At minimum: confirmation of the 1.0 target (question 2), the Certify route (question 3), and whether the 0.18.2 findings are worth reporting and where (question 4). Everything else has a workable default.

1.5 Notes during the session

Q1 _____

Ver. _____

Q7 _____

Issues _____

Other _____

_____

After the session, record the answers in docs/findings/findings.tex (the Open Questions section) and update the planner in Part 4 where an answer changes it.

Part 2 Start to end: running the project from a clean machine

Follow this top to bottom on a machine that has never run the project. Every command has been run on the reference machine (macOS, Apple Silicon); Windows differences are noted where they exist. Budget two to three hours the first time, most of it image downloads.

2.1 What runs, and why each piece is there

Piece	Where it listens	Why it is needed
OpenID conformance suite 5.3.1	https://localhost.emobix.co.uk:8443	The examiner. It plays the wallet (verifier tests) or the client (issuer tests) and checks the component against the spec.
Inji Verify 1.0.0-alpha.1	UI :3000, API :8080 (path /v1/verify)	The verifier under test. Happy-flow passes on it as of 2 October.
Inji Certify 0.14.0	not yet running	The issuer under test (Part 4, weeks 3-4).
ngrok tunnel	a *.ngrok-free.dev address	Mandatory. Verify identifies itself with a did:web DID resolved over public HTTPS, and bakes its public address into the response_uri the suite must reach from inside its own container.
The runner (this repo)	a command, not a server	Drives the suite's REST API and Verify's API, normalises results, applies the gate.

2.2 Step 1 -- Prerequisites

Git, Docker Desktop, Java 21, Maven 3.9+, Python 3.11+, ngrok. Check:

```
git --version; docker --version; java -version; mvn -version
python3 --version; ngrok version
```

macOS: `brew install git openjdk@21 maven python3 ngrok`, plus Docker Desktop from docker.com. Windows: winget for the same set, Docker Desktop with WSL 2, and once only: `git config -global core.autocrlf input` so shell scripts keep Unix line endings.

Java is **21**, not 11: the Inji api-test projects require it, whatever older MOSIP documentation says.

2.3 Step 2 -- The working directory

The runner expects these six folders side by side under one root. The folder names matter: the runner finds Inji Verify's UI configuration at ../inji-verify.

```
mkdir -p ~/projects/MOSIP && cd ~/projects/MOSIP
git clone https://github.com/shard-c6/mosip-decode-ps1.git
git clone --depth 1 https://github.com/mosip/inji-verify.git
git clone --depth 1 https://github.com/mosip/inji-certify.git
git clone --depth 1 https://github.com/mosip/mosip-functional-tests.git
git clone https://gitlab.com/openid/conformance-suite.git
git clone https://gitlab.com/openid/conformance-suite-automated-testing-tutorial.git
```

Then the **target version**, Inji Verify 1.0, as a seventh folder: a worktree of the inji-verify clone at the published 1.0 tag. The 0.18.2 checkout stays as it is, because it reproduces the earlier evidence.

```
git -C inji-verify fetch --depth 1 https://github.com/mosip/inji-verify.git \
    tag v1.0.0-alpha.1
git -C inji-verify worktree add --detach ../inji-verify-1.0 v1.0.0-alpha.1
```

mosip-decode-ps1 is ours. The Java work lands on branches of our forks of inji-verify and inji-certify (Shardul's forks at github.com/shard-c6; Mukta has write access to both).

2.4 Step 3 -- ngrok, once

Create a free ngrok account, then `ngrok config add-authtoken <token>` and claim the account's free **static** domain in the dashboard. Without a static domain the hostname changes on every restart and Step 4 must be redone each time. The domain is per-developer: Shardul's is sphere-dust-sandlot.ngrok-free.dev; Mukta needs her own.

2.5 Step 4 -- Stack settings

Since 27 October the whole stack -- the conformance suite and Inji Verify 1.0 -- runs from this repository's docker/docker-compose.yml. Inji Verify's files need no manual edits: the two it needs fixed (the F-10 database script, and a conformance credential in its UI config) are kept in docker/inji-verify/, with their provenance recorded there.

One file to create:

```
cp docker/.env.example docker/.env
# then set PUBLIC_HOST=YOUR-DOMAIN.ngrok-free.dev   (hostname only)
```

docker/.env is gitignored. Set NGROK_AUTHTOKEN there only if you want the stack to run ngrok itself (the tunnel profile); otherwise run ngrok on the host.

2.6 Step 5 -- Point the runner at your machine

5a. A signing key for the suite's test credential. Private, so it lives in the gitignored configs/keys/. Generated with the suite's own script:

```
cd ~/projects/MOSIP/mosip-decode-ps1
python3 ../conformance-suite/scripts/generate-vp-test-cert.py \
    --hostname YOUR-DOMAIN.ngrok-free.dev \
    --output configs/keys/vp-signing-jwk.json --ca-output configs/keys/vp-test-ca.pem
```

5b. Your own settings. configs/runner.json is shared and committed. Anything that is yours goes in configs/runner.local.json, which git ignores and the runner deep-merges on top, matching components by name. The client identifier must use the redirect_uri: prefix with *your* response URI, and a distinct alias keeps two developers' plans apart:

```
{ "components": [ { "component": "inji-verify", "alias": "injiverify-YOURNAME",
    "verifierRequest": { "clientId":
      "redirect_uri:https://HOST/v1/verify/v2/vp-submission/direct-post"
} } ] }
```

HOST is your ngrok domain, e.g. sphere-dust-sandlot.ngrok-free.dev.

Install the runner's one dependency: `python3 -m pip install -r requirements.txt`.

2.7 Step 6 -- Start the stack

6a. Docker Desktop. Start it and wait for the engine: `docker info`.

6b. The tunnel, in its own terminal -- or pass -tunnel below instead:

```
ngrok http --url=https://YOUR-DOMAIN.ngrok-free.dev 3000
```

6c. Everything else, one command. It starts the suite and Inji Verify and waits until each answers, including over the public hostname:

```
scripts/stack.sh up verify            # or: scripts/stack.sh up verify --tunnel
```

The underlying Compose profiles, should you need them directly (run in docker/): no profile starts the suite alone; -profile verify adds Inji Verify; -profile tunnel adds ngrok; -profile certify is planned. Only one copy of the stack runs at a time -- it uses ports 8443, 8444, 3000 and 8080.

2.8 Step 7 -- Check everything is healthy

All three must answer before a run is worth attempting:

```
curl -k https://localhost.emobix.co.uk:8443/api/currentuser    # JSON, dev user
curl http://localhost:8080/v1/verify/did.json                  # Verify, local
curl https://YOUR-DOMAIN.ngrok-free.dev/v1/verify/did.json     # Verify, public
```

The last is the one that matters most: if the public DID document does not resolve, nothing downstream can work. The path is /v1/verify/did.json -- there is no .well-known, because the DID has path segments.

2.9 Step 8 -- Run the unit tests

Needs no Docker. Normalisation is tested against the real Day 1 log, so these also check that the suite's field names have not drifted from our contract.

```
cd ~/projects/MOSIP/mosip-decode-ps1 && python3 -m pytest tests/ -q     # expect 63 passed
```

2.10 Step 9 -- Run conformance

```
./run-conformance.sh --component verify                  # the full verifier plan
./run-conformance.sh --component verify \
    --only oid4vp-1final-verifier-happy-flow -v          # one module, verbose
./run-conformance.sh --component verify --output scratch/run.json
./run-conformance.sh --component verify \
    --config configs/runner-0.18.json                    # 0.18.2: earlier evidence
./run-conformance.sh --component certify                 # once Certify is configured
./run-conformance.sh --combined                          # every configured component
```

Useful options: -nonce-mode sdk|service (Part 3, 29 September, for why this exists), -policy absolute-pass-rate, -no-gate-exit for exploratory runs only.

Exit status is the verdict, and CI keys on it:

```
0  gate passed -- nothing regressed against the baseline
1  gate failed -- a regression, or the harness could not execute a module
2  configuration error -- nothing was run
```

2.11 Step 10 -- Verify the results

A run that finished is not yet a run that is right. Check, in order:

1. **The gate summary** printed at the end. "Harness could not execute" means a fault in us or the environment, never a conformance result.

2. **Each module's handoff.** In the output JSON, verifier modules carry a handoff block: delivered true, httpStatus 302, and secondsRemaining well above zero. The request expires 300 s after issue.
3. **Checks against the baseline.** Every non-passing check carries a findingRef (F-01 ... F-09) when it matches a known finding. A failing check with no findingRef is new and needs reading before anything else.
4. **The suite's own view** at the logUrl for any module in doubt.
5. **Archived logs** under logs/<date>/. These are the JSON the runner fetched and are *unsigned*. For anything that will be reported upstream, export the signed log from the suite (*Download all Logs*) and commit the .json and .sig together.

2.12 Step 11 -- Record what the run showed

- A new or changed finding: a \finding block in docs/findings/findings.tex, tagged CONFIRMED or UNVERIFIED, with the log line quoted and the spec section the suite cited. Nothing goes upstream while UNVERIFIED.
- A day's work: a dated subsection in the findings Day Log -- objective, runs with test IDs, decisions with reasons, status.
- A module whose result is now understood: an entry in configs/contract/expected-failures.json with its reason and an evidence path. An expected failure without a reason is a silenced test.
- Every run worth keeping: a row in logs/README.md.

Rebuild the findings PDF:
`cd docs/findings && pdflatex findings.tex && pdflatex findings.tex`.

2.13 Step 12 -- Commit

Stage by path, never blindly: `git status -short -untracked-files=all` first. Subject in the imperative; body explaining why. **No assistant attribution lines of any kind** -- commits are made under our own names. Push to master on shard-c6/mosip-decode-ps1.

2.14 Step 13 -- Shut down

```
scripts/stack.sh down
```

Then stop ngrok if it runs on the host (Ctrl-C). down keeps the data volumes, so the suite's plans and logs survive. **Never** add -v unless you mean to delete every plan and log the suite holds.

2.15 Step 14 -- Ending the project

The final week, in order. Detail and dates in Part 4.

1. Fresh-clone test: follow only this Part 2 on a machine that has never run the project. Anything missing gets fixed here, not explained later.
2. Final combined run, recorded; signed logs exported and committed.
3. Upstream: file confirmed findings as issues against mosip/inji-verify, and offer the fixes as pull requests from our forks.
4. Submission package: public repo with a clear README; the 1-2 page document or 5-7 slide pitch; demo video; the Docker Compose, test plan configuration, one-command runner and GitHub Actions workflow the problem statement lists.
5. Submit **by email to decode@mosip.io** -- *not* with the Unstop page's submit button, which the organisers say is only a placeholder. Source code in the GitHub repository; every other deliverable in a shared cloud folder (e.g. Google Drive); the email gives the team name, the problem statement and those URLs, and every link must be accessible. Target Sunday 1 November; the deadline is Monday 2 November.

6. The guidelines' mandatory items: source code, working prototype or demo, slide deck, 2-5 minute video, usage and installation instructions, API documentation where applicable, a git repository with history, and *each* student's college ID.
7. Hand-over: MOSIP may request the code be contributed. Keep the repository clean enough that this is a formality.
8. Shut the stack down (Step 13).

2.16 Troubleshooting -- problems that have already cost us time

Symptom	Cause and fix
Verify returns HTTP 400 to vp-session-request	The presentation definition has no id. The runner builds the UI's envelope; if this appears, check the credential name in runner.json matches config.json exactly.
A module fails at request_uri	The credential uses did (by reference). Redo Step 4b.
Failure that looks like an expired request	Requests die 300 s after issue. The runner refuses to deliver with under 30 s left; manually, check liveness with curl .../v1/verify/vp-request/<id> first.
Verify behaves as if unconfigured	Environment variables are read at container *creation*. After editing docker-compose.yml: down, then up -d. restart is not enough.
Suite "stopped" but still running	A plain docker compose down in that folder targets the wrong file. Always -f docker-compose-prebuilt.yml.
DID document 404	The path is /v1/verify/did.json; no .well-known.
Gate passed but nothing ran	Fixed on 29 September: a harness error now fails the gate. If you ever see this again, it is a regression in the gate.
Unit tests pass, CLI crashes on import	Fixed on 29 September by import tests. Run the full pytest tests/, not a subset.
Browser warns the suite is not secure	Expected: self-signed certificate on a hostname that resolves to 127.0.0.1. Applies only because it is your own machine.

Part 3 Record of every step taken

What we did, in order, and what each step produced. Detail, evidence and quoted logs for every item are in docs/findings/findings.tex; this is the index to it.

3.1 16-20 September -- registration and pre-kickoff groundwork

1. Registered as a team for MOSIP Decode 2026 (confirmed 17 September) and chose Problem Statement 1.
2. Cloned the five upstream repositories side by side under /projects/MOSIP.
3. Started the conformance suite 5.3.1 locally from prebuilt images. Established that its local profile needs no API token.
4. Started Inji Verify 0.18.2 (three containers). Found the unsubstituted VERIFY_SERVICE_PROXY_FOR_LOCALHOST placeholder and replaced it with an ngrok hostname; found that did:web makes a public tunnel mandatory.
5. Created the verifier test plan lN7C4DH1HvYmc (OID4VP 1.0 Final alpha, sd_jwt_vc / redirect_uri / url_query / plain_vp / direct_post).
6. Ran the happy-flow module by hand three times. Two runs were discarded, each for a reason we could prove: an expired request, then a variant mismatch fixed by switching the SD-JWT credential to pre_registered. Both had looked like Inji Verify defects. The third, MtzpWoD2dVgtic0, is the baseline.
7. Catalogued nine findings (F-01 to F-09), eight confirmed. Headline: Verify implements Presentation Exchange, not DCQL, so no 1.0 Final module can pass.
8. Wrote build-oid4vp-uri.py, the first piece of tooling; scaffolded the repository with setup, runbook, plan and findings documents.

3.2 23 September -- official kickoff: plan, roles, contract

1. Re-anchored the plan: the 20 September work predated the event.
2. Agreed roles by skill rather than by language. Mukta: platform and delivery. Shardul: conformance engine and evidence. Estimated split 49/51.
3. Chose to put the api-testrig work on our forks of inji-verify and inji-certify, shaped as upstream pull requests. api-test lives in those repositories, not in mosip-functional-tests.
4. Wrote the runner-to-testrig contract (docs/CONTRACT.md) with a fixture generated from the real Day 1 log, so the Java side need not wait for the runner.
5. Traced F-01/F-02 to btoa(Date.now()) in inji-verify-sdk, and found that verify-service already generates a sound nonce but only as a fallback. Drafted the upstream issue.

3.3 24 September -- the runner

1. Resolved the week's blocking unknown from the suite's source: a verifier module receives its authorisation request by an ordinary HTTP GET to the authorization_endpoint it exposes. No browser is needed.
2. Found the suite's built-in path for the screenshot requirement: a browser automation entry against an HTML stand-in page the suite serves itself.
3. Built the runner: configuration, suite client, Verify client, handoff, normalisation, gate, orchestrator, CLI, and run-conformance.sh.
4. 24 unit tests; one caught a defect in the gate before any live run.

3.4 29 September -- first live runs

1. Found and fixed two request defects before running, by reading Verify's DTO: no presentation definition was sent, and no nonce -- which would have made F-01 and F-02 silently vanish from automated runs.
2. Run 1 failed with HTTP 400: the definition lacked an id. Traced to the envelope the UI builds in vpVerificationState.ts; the runner now builds the same one.

3. Run 1 also exposed the most serious defect: the gate reported PASSED having measured nothing. Harness errors now always fail the gate.
4. Run 2, hcuWEvxGbvRgt7e: first fully unattended run. Matches the manual baseline check for check.
5. Run 3, Q9MaDZyfpJhjiDp: controlled experiment on the nonce source. Confirmed the F-01/F-02 root cause.
6. Emailed Mukta the directory setup and project status. 50 unit tests.

3.5 30 September -- deadline found, this handbook

1. Found the real deadline on the official event page: **26 October 2026, 11:59 PM IST**, not the 21 October we had assumed. The plan gains five days.
2. Found the Weekly Connect schedule -- Wednesdays, 5:00-6:00 PM IST, from today.
3. Wrote this handbook: mentor brief, start-to-end guide, this record, the planner.

3.6 2 October -- the kickoff recording, and the 1.0 line

1. Worked through the transcript of the 23 September kickoff webinar, which we had not attended live. Correction: earlier documents said it had not taken place; it had.
2. Took from it: the mandatory submissions are source code, a working prototype, a slide deck and a video demo; the Docker Compose must bring up each module alone or both together; CI's purpose is catching non-conformance per change.
3. Followed up a passing mention of "DCQL requests" and found the Inji 1.0 line: release-1.0.x in Verify and Certify, published as 1.0.0-alpha.1. On it, DCQL exists and the nonce is random -- F-01, F-02 and F-03 do not apply.
4. Put the F-01/F-02 upstream issue on hold; scoped every finding to 0.18.2; decided to move the baseline to 1.0.0-alpha.1.
5. Taught the runner the 1.0 API (DCQL, /v2/vp-session-request, the 1.0 SDK's behaviour). Resolved F-05 as our own artefact on the way.
6. Five runs on 1.0.0-alpha.1, each changing one thing. Found F-10 (the Compose database script breaks every submission) and F-11 (sample config uses a superseded format). Established what a passing run needs.
7. **Happy-flow passed: FINISHED / REVIEW, 35 checks, 0 failures, 0 warnings, fully unattended** -- including the screenshot. The gate now protects a real pass.
8. Ran the full plan. Added REVIEW resolution: the harness fetches Inji Verify's own verdict, because the suite cannot see whether a forged credential was rejected. Result: 9 of 11 modules exercised, all consistent with conformance.

3.7 26 October -- deadline extended; the api-testrig class

1. No work between 2 and 26 October. The organisers extended the deadline to 2 November and confirmed submission is by email, not the Unstop button.
2. Built Inji Verify's api-test on release-1.0.x: apitest-commons 1.5.0 resolves from Maven Central.
3. Wrote OpenIDConformanceTest: each conformance module becomes a TestNG result in the testrig's own report, plus one for the gate. Checked on the real 11-module run (12 of 12 pass, all named in MOSIP's EmailableReport) and on a doctored result with a regression and an accepted forged credential (exactly those and the gate fail). Pushed to shard-c6/inji-verify, branch feature/openid-conformance-test.
4. The same class in Inji Certify's api-test (shard-c6/inji-certify, same branch name): 16 of 16 on the recorded issuer run; a doctored regression fails exactly that module and the gate; unconfigured, one skipped test.
5. The combined run, ./run-conformance.sh -combined: both plans, 26 modules, one result and one gate. Each component identical to its standalone run; gate passes. The one result file gives 12 of 12 in Inji Verify's testrig and 16 of 16 in Inji Certify's.

Part 4 Planner, to 2 November

4.1 Re-plan, 26 October: the final week

The deadline moved to 2 November. No work happened between 2 and 26 October, so the week-by-week plan below records what had been intended and is now superseded by this table. Mukta's assigned work (Compose, the Java class, CI, the Certify environment) had not started; Shardul takes all of it on. Scope is cut to what one person can finish well: Inji Verify end to end, with Inji Certify time-boxed to one day.

Day	Work	Status
Mon 26 Oct	OpenIDConformanceTest in Inji Verify's api-test (fork branch feature/openid-conformance-test); runner -endpoint	[x] done
Tue 27 Oct	Docker Compose with per-module profiles; GitHub Actions running the unit and contract tests -- done a day early, 26 Oct	[x] done
Wed 28 Oct	Inji Certify, **time-boxed to one day**: issuer support; one issuer module if the stack comes up -- done early, 26 Oct: full 15-module plan run, 3 pass, 4 findings	[x] done
26 Oct (extra)	OpenIDConformanceTest in Inji Certify's api-test; combined run of both components, gated as one	[x] done
Thu 29 Oct	Architecture diagram; README; runner API documentation; signed evidence exports	[ ]
Fri 30 Oct	Slide deck (problem, solution, stack, audience, impact); video, 2-5 min	[ ]
Sat 31 Oct	Fresh-clone test; fixes; contribution record made accurate	[ ]
Sun 1 Nov	Cloud folder; college IDs; **submission email to decode@mosip.io**	[ ]
Mon 2 Nov	Buffer only -- deadline	

Cut: a real Inji Certify pass, the HAIP plan, the two request-uri modules, result diff, the badge.

S = Shardul (conformance engine and evidence). **M** = Mukta (platform and delivery). Tick boxes as work lands; move a slipped item forward rather than deleting it, so the planner stays an honest record.

4.2 Fixed points

Wed 30 Sep	Weekly Connect (first session)
Sun 4 Oct	End of week 2: verifier plan baselined on 1.0.0-alpha.1; api-test builds
Wed 7 Oct	Weekly Connect -- mentor brief, Part 1
Sat 10 Oct	**Certify decision**: is a Certify 1.0 stack running? If not, fall back (Risks)
Sun 11 Oct	End of week 3: runner output lands in the api-testrig report
Wed 14 Oct	Weekly Connect
Sun 18 Oct	End of week 4: **minimum viable submission**
Tue 20 Oct	Nothing new or risky starts after today
Wed 21 Oct	Weekly Connect -- show the working build if possible
Fri 16 Oct	**Shardul's part complete** (estimate; see below)
Fri 23 Oct	**First submission** (safety net)
Mon 26 Oct	*Original deadline -- extended to 2 November*
Sun 1 Nov	**Submission email** to decode@mosip.io (target)
Mon 2 Nov	**Final submission deadline**

4.3 Week 2 remainder -- Wed 30 Sep to Sun 4 Oct: move to the 1.0 line

Re-planned on 2 October. Wednesday and Thursday's items did not happen and are carried forward; the discovery of the 1.0 line changes what "finish the verifier side" means.

Day	S	M
Wed 30	[ ] Weekly Connect (attendance not recorded)	[x] Directory setup email received (29 Sep) [x] Sent the kickoff recording link (30 Sep)
Fri 2 Oct	[x] Kickoff transcript worked through [x] Found the 1.0 line; findings scoped; issue on hold [x] Bring up Verify 1.0.0-alpha.1; run happy-flow -- **passes** [x] F-10 found; upstream issue drafted	[ ] Accept the two fork invitations; claim an ngrok domain [ ] Build apitest-commons; api-test in inji-verify **Build api-test from release-1.0.x**, not master -- the target changed
Sat 3 Oct	[x] Full 1.0 Final plan on alpha.1 -- done early, 2 Oct: 9 of 11 exercised and consistent, 2 n/a to the variant; baseline for all 11 [ ] Make nonceMode follow the version under test	[ ] api-test in inji-certify (release-1.0.x) [ ] Write down the TestNG/Extent result shape
Sun 4 Oct	[x] Screenshot mechanism -- confirmed by run, 2 Oct [ ] Re-check F-05, F-06, F-07, F-09 on 1.0	[ ] Review CONTRACT.md; OpenIDConformanceTest skeleton [ ] Read the 1.0 compose; sketch per-module profiles

Exit check: ./run-conformance.sh -component verify runs all 12 modules against 1.0.0-alpha.1 with a baseline entry for each; api-test builds in both repositories on release-1.0.x.

4.4 Remaining effort -- Shardul's part (estimate, 2 October)

Made after the verifier plan was completed on Inji Verify 1.0, and after checking what the issuer side needs: the suite's OID4VCI issuer plan has 20 modules (48 in the HAIP plan), and Inji Certify 1.0 supports the pre-authorized_code grant, so the issuer plan can be driven without scripting a browser login. Hours are focused working hours.

Task	Hours	Needs Certify?
Issuer support in the runner: pre-authorized flow, issuer plan config, tests	8-12	no
Certify integration, run by run, as Verify was on 2 October	8-14	**yes**
Combined run across both components; consolidated report	3-4	partly
Evidence: signed exports; F-10 filed upstream with its fix	3	no
Docs: architecture, automation, self-certification guide	8	no
Share of the slide deck / write-up	4-6	no
Polish: README, findings PDF margins	3	no
Total	**37-50**	

At about 5 focused hours a day: **8-10 working days**, finishing around **14-16 October**, ten days before the deadline. The one item that can push this out is Certify integration, which waits on a running Certify 1.0 stack (Mukta). Everything else on this list can proceed without it.

4.5 Week 3 -- Mon 5 to Sun 11 Oct: the two halves meet

Day	S	M
Mon 5	[ ] Issuer support: read the suite's issuer modules and Certify's credential-offer API; design the pre-authorized handoff (≈5h)	[ ] Unified Docker Compose with **per-module profiles**: Verify, Certify, or both, each with the suite -- the kickoff's stated requirement
Tue 6	[ ] Issuer support: code and tests against fixtures; issuer plan config template (≈5h)	[ ] Compose: solve cross-project networking and the public hostname
Wed 7	[ ] Weekly Connect -- brief in Part 1 [ ] Finish issuer support (≈2h)	[ ] OpenIDConformanceTest in inji-verify's api-test, reading real runner output
Thu 8	[ ] Combined run: consolidated output across components (≈3-4h)	[ ] Inji Certify environment: keystore, DID endpoint, auth server, keys -- day 1
Fri 9	[ ] Signed exports; file F-10 upstream, per the mentor answer on where (≈3h)	[ ] Certify environment -- day 2
Sat 10	[ ] Architecture and automation docs (≈4h) [ ] **Certify decision** with Mukta	[ ] Certify environment -- day 3, or fall back (see risks)
Sun 11	[ ] Joint integration session: runner output into the Extent report, end to end. Buffer otherwise.	

Exit check: a Verify run's results appear as TestNG pass/fail/skip in Inji Verify's own api-testrig report.

4.6 Week 4 -- Mon 12 to Sun 18 Oct: complete the mandatory set

Day	S	M
Mon 12	[ ] Certify integration: issuer plan live, one change per run (≈4-7h)	[ ] Fold Certify into the unified compose
Tue 13	[ ] Certify integration continued; Certify baseline (≈4-7h)	[ ] OpenIDConformanceTest in inji-certify's api-test
Wed 14	[ ] Weekly Connect [ ] Combined run end to end	[ ] GitHub Actions workflow, first version
Thu 15	[ ] Self-certification guide; finish docs (≈4h)	[ ] Result diff between runs; consolidated report and badge
Fri 16	[ ] Share of the deck / write-up (≈4-6h) **Target: Shardul's part complete**	[ ] Actions: fail the build on a regression, demonstrably
Sat 17	[ ] Polish: README, findings PDF margins (≈3h)	[ ] Setup, troubleshooting and CI documentation
Sun 18	[ ] **Minimum viable submission check**: all three invocations work and land in the api-testrig output.	

4.7 Week 5 -- Mon 19 to Mon 26 Oct: submit

Day	S	M
Mon 19	[ ] Documentation complete; README final	[ ] Fresh-clone test using only Part 2
Tue 20	[ ] Fix whatever the fresh-clone test found	[ ] Fix whatever the fresh-clone test found
Wed 21	[ ] Weekly Connect: show the build; collect feedback	
Thu 22	[ ] 1-2 page document or 5-7 slide pitch	[ ] Demo video, end to end
Fri 23	[ ] **First submission.**	
Sat 24-Sun 25	[ ] Final combined run recorded; signed logs [ ] File upstream issues	[ ] Final CI run green; video re-cut if needed
Mon 26	[ ] **Final submission** -- done by 6:00 PM IST; the deadline is 11:59 PM. Then shut down (Part 2, Step 13).	

4.8 Dependencies between us

This	needs	Blocked meanwhile?
OpenIDConformanceTest (M)	runner output (S)	No -- the committed fixture is real output shape
Issuer-plan runs (S)	Certify environment (M)	Yes, from week 4; issuer support is built in week 3 regardless
Combined run (both)	Certify environment and issuer support	Yes -- the reason Certify starts in week 3
GitHub Actions (M)	unified compose (M) and runner (S)	Partly -- the unit tests can run in CI from day one
Upstream issues (S)	mentor answer on channel	No -- default is a GitHub issue

4.9 Risks, and what we do if they land

Risk	Response
Certify cannot run locally	Decided on Sat 10 Oct. Either Shardul brings the Certify 1.0 stack up himself (adds 1-2 days to his part), or fall back: submit Verify end to end, the issuer support in the runner, and a documented Certify path. Per-module conformance for one component satisfies the mandatory tasks.
Mukta's start slips	No word on setup progress as of 2 October. The fixture means the Java class can start the moment setup is done; if api-test is not building by Fri 2 Oct, pair on it that weekend.
Neither of us writes Java	The class is kept small and logic-free by design. Escalate by mid-week 3 rather than absorb it quietly.
Screenshot mechanism fails	Only affects modules that get past DCQL. Fallback: one documented manual upload, stated plainly rather than hidden.
Mentor answers change the target	Plan and variants are configuration; a version change costs an edit and a re-run, not a rewrite.